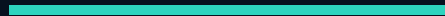


Օբյեկտային ուղղվածությամբ ծրագրավորում

Դաս, օբյեկտ և դասի բաղադրիչ բաժիններ



Դասի ուսումնառության արդյունքները

- Բացատրել դասի և օբյեկտի տարբերությունը
- Սահմանել class և նկարագրել դրա անդամները
- Կիրառել private, public և protected բաժինները
- Ստեղծել օբյեկտներ և կանչել դրանց մեթոդները
- Կազմակերպել տվյալների անվտանգ փոփոխումը

Օբյեկտային ուղղվածությամբ մոտեցումը

Օբյեկտը միավորում է վիճակը և այդ վիճակի հետ աշխատող գործողությունները:

ՎԻՃԱԿ

Տվյալները նկարագրում են վիճակը: Մեթոդները սահմանում են վարքագիծը:

ՎԱՐՔԱԳԻԾ

Օրինակ՝ հաշվի մնացորդը վիճակ է, deposit() մեթոդը՝ վարքագիծ

Դաս և օբյեկտ

ԴԱՍ

Նոր տիպի ընդհանուր
նկարագրություն: Սահմանում է
անդամներն ու հասանելիության
կանոնները:

ՕԲՅԵԿՏ

Դասի հիման վրա ստեղծված կոնկրետ
փոփոխական: Ունի սեփական
վիճակը:

class-ի հիմնական շարահյուսությունը

```
class Student {  
private:  
    string name;  
    double grade = 0;  
  
public:  
    void setGrade(double value);  
    double getGrade() const;  
};
```

- class-ից հետո գրվում է դասի անունը
- Մարմինը սահմանափակվում է ձևավոր փակագծերով
- Բաժինների անուններից հետո դրվում է երկու կետ
- Դասի նկարագրությունն ավարտվում է կետ-ստորակետով

Դասի բաղադրիչները

Տվյալների անդամներ

Նկարագրում են օբյեկտի վիճակը:

name
grade
balance

Անդամ մեթոդներ

Կատարում են գործողություններ և ստուգումներ:

setGrade()
deposit()
printInfo()

Հասանելիության բաժինները

Բաժին	Որտեղից է հասանելի	Հիմնական նպատակ
private	դասի ներսից	տվյալների պաշտպանություն
public	ծրագրի հասանելի մասերից	օբյեկտի արտաքին կառավարում
protected	դասից և ժառանգ դասերից	ժառանգման համար հասանելի անդամներ

class-ի անդամները լռելյայն private են

class և struct լռելյայն հասանելիությունը

```
class Student {  
    int course; // private  
};
```

```
struct Student {  
    int course; // public  
};
```

Երկուսն էլ կարող են պարունակել տվյալներ և մեթոդներ

Օբյեկտի ստեղծում և մեթոդի կանչ

```
Student firstStudent;  
Student secondStudent;  
  
firstStudent.setGrade(18);  
secondStudent.setGrade(15);  
  
cout << firstStudent.getGrade();
```

firstStudent.setGrade(18)

- կետից առաջ՝ օբյեկտ
- կետից հետո՝ public մեթոդ
- յուրաքանչյուր օբյեկտ ունի սեփական վիճակը

private տվյալների վերահսկվող փոփոխում

```
class Student {  
    private:  
        double grade = 0;  
  
    public:  
        bool setGrade(double value) {  
            if (value < 0 || value > 20)  
                return false;  
            grade = value;  
            return true;  
        }  
};
```

private դաշտը չի
փոխվում սխալ արժեքով

Setter մեթոդը պահպանում է
օբյեկտի վավեր վիճակը

Getter և setter մեթոդներ

```
void setName(const string& value) {  
    if (!value.empty()) name = value;  
}  
  
const string& getName() const {  
    return name;  
}  
  
double getGrade() const {  
    return grade;  
}
```

- Setter-ը վերահսկում է նոր արժեքը
- Getter-ը վերադարձնում է տվյալը
- Միայն ընթերցող getter-ը նշվում է const
- Ամեն դաշտի համար setter ստեղծելը պարտադիր չէ

Մերօղը դասից դուրս

```
class Rectangle {  
private:  
    double width = 0;  
    double height = 0;  
public:  
    double area() const;  
};
```

```
double Rectangle::area() const {  
    return width * height;  
}
```

Rectangle::area

const անդամ մեթոդ

```
double getBalance() const {  
    return balance;  
}  
  
void printAccount(  
    const BankAccount& account) {  
    cout << account.getBalance();  
}
```

**Մեթոդը չի փոխում
օբյեկտի վիճակը**

const օբյեկտի համար կարելի է
կանչել միայն const մեթոդներ

this ցուցիչը

```
class Student {  
private:  
    string name;  
public:  
    void setName(const string& name) {  
        if (!name.empty()) {  
            this->name = name;  
        }  
    }  
};
```

this->name

Ընթացիկ օբյեկտի name
դաշտը տարբերվում է մերձողի
name պարամետրից

Միևնուշյն դասի անկախ օբյեկտներ

```
Counter first;  
Counter second;  
  
first.increment();  
first.increment();  
second.increment();  
  
cout << first.getValue(); // 2  
cout << second.getValue(); // 1
```

first.value = 2

second.value = 1

Մեթոդը փոխում է միայն
ընթացիկ օբյեկտը

Օբյեկտը \$ուևկցիայի պարամետր և արդյունք

```
void printStudent(  
    const Student& student) {  
    cout << student.getName() << " "  
        << student.getGrade();  
}  
  
Student betterStudent(  
    Student first, Student second) {  
    if (first.getGrade() >= second.getGrade())  
        return first;  
    return second;  
}
```

- const հղումը չի ստեղծում պատճեն և չի փոփոխում օբյեկտը
- Արժեքով փոխանցումը ստեղծում է անկախ պատճեն
- \$ուևկցիան կարող է վերադարձնել դասի օբյեկտ

Ամբողջական BankAccount դասը

```
class BankAccount {  
private:  
    string owner;  
    double balance = 0;  
public:  
    void setOwner(const string& value) {  
        if (!value.empty()) owner = value;  
    }  
  
    bool deposit(double amount) {  
        if (amount <= 0) return false;  
        balance += amount;  
        return true;  
    }  
};
```

```
    bool withdraw(double amount) {  
        if (amount <= 0 || amount > balance)  
            return false;  
        balance -= amount;  
        return true;  
    }  
  
    const string& getOwner() const {  
        return owner;  
    }  
  
    double getBalance() const {  
        return balance;  
    }  
};
```

BankAccount օբյեկտի օգտագործումը

```
BankAccount account;  
account.setOwner("Ani");  
  
account.deposit(50000);  
  
if (!account.withdraw(12000)) {  
    cout << "Operation failed";  
}  
  
cout << account.getOwner() << " "  
    << account.getBalance();
```

- main-ը չի դիմում balance դաշտին
- deposit և withdraw մեթոդները ստուգում են տվյալները
- օբյեկտը պահպանում է վավեր վիճակը

**Վերջնական մնացորդ՝
38000**

Ամփոփում և ստուգիչ հարցեր

- Ի՞նչ տարբերություն կա դասի և օբյեկտի միջև
- Որո՞նք են դասի տվյալների անդամներն ու մեթոդները
- Ի՞նչ նպատակ ունեն private և public բաժինները
- Ինչո՞ւ է setter-ը ստուգում նոր արժեքը
- Ի՞նչ է նշանակում մեթոդի վերջում գրված const-ը
- Ե՞րբ է կիրառվում this ցուցիչը